



Institute of Information Technology
University of Dhaka

Career Talk

AI-Powered Career Guidance & Mentorship Platform

Technical Report
Document of SPL-03 [SE-801]

Submitted By:

Ibne Bin Rafid
Exam Roll: 2322208
BSc in Software Engineering

Supervised By:

Dr. Zerina Begum
Professor
Institute of Information Technology
University of Dhaka

February 2026

Certificate of Approval

This is to certify that the project titled “**Career Talk: AI-Powered Career Guidance & Mentorship Platform**” has been successfully completed by **Ibne Bin Rafid** (Exam Roll: 2322208) in partial fulfilment of the requirements for the degree of **Bachelor of Science in Information Technology** from the **Institute of Information Technology, University of Dhaka**.

Dr. Zerina Begum

Professor

Institute of Information Technology

University of Dhaka

Supervisor

Declaration

I hereby declare that this project titled “**Career Talk: AI-Powered Career Guidance & Mentorship Platform**” submitted for the fulfilment of the requirements of **SPL-03 [SE-801]** is my original work and has not been submitted elsewhere for any academic degree or diploma.

All the external information and resources used in this project have been duly acknowledged and cited.

Ibne Bin Rafid

Exam Roll: 2322208

Date: March 1, 2026

Acknowledgement

I wish to express my heartfelt gratitude to my supervisor, **Dr. Zerina Begum**, Professor, Institute of Information Technology, University of Dhaka, for her valuable guidance, encouragement, and support throughout the development of this project. Her insights played a major role in shaping the vision and direction of Career Talk.

I am grateful to the **Institute of Information Technology, University of Dhaka**, for providing the academic environment and resources that made this project possible.

I also want to thank the open-source community for the excellent tools and libraries, including Django, React, LangChain, FAISS, and many others, that form the backbone of this platform.

Lastly, I thank my family and friends for their constant support and encouragement during this journey.

Abstract

Career Talk is an AI-powered career guidance and mentorship platform that aims to make quality career advice more accessible to South Asian students, fresh graduates, and early-career professionals. It tackles the gap between traditional career counselling, which is often limited, expensive, and inaccessible, and the dynamic, personalized guidance that modern job seekers need.

The platform is built around three central pillars. First, **AI-driven Career Planning** uses a multi-agent LLM pipeline built with LangChain and Google Gemini/Groq, combined with Retrieval-Augmented Generation (RAG) on a FAISS vector database, to produce personalized career roadmaps. Second, **Real-time Mentorship** lets users discover verified mentors, schedule sessions, and communicate through WebSocket-powered messaging and Agora-based video meetings. Third, an **Agentic AI Chatbot** delivers context-aware, interactive career guidance by drawing on the user's profile data and domain knowledge.

The technology stack includes Django REST Framework on the backend and React.js with Tailwind CSS on the frontend, supported by PostgreSQL, Redis, Celery, and Docker. The architecture features JWT authentication with token rotation, real-time WebSocket communication, asynchronous task processing, and a responsive interface with both dark and light themes.

This report documents the complete software engineering lifecycle, covering requirements analysis and system design through implementation, testing, and deployment, and serves as a comprehensive final deliverable for SPL-03.

Keywords: AI Career Guidance, LangChain, RAG, FAISS, Django REST Framework, React.js, WebSocket, Agora Video, Mentorship Platform, Multi-Agent AI System.

Contents

List of Abbreviations	11
1 Introduction	12
1.1 Background and Motivation	12
1.2 Problem Statement	12
1.3 Proposed Solution	13
1.4 Objectives	13
1.5 Scope	13
1.6 Report Organization	14
2 Literature Review and Related Work	15
2.1 Existing Career Guidance Platforms	15
2.2 Large Language Models in Career Guidance	15
2.3 Retrieval-Augmented Generation (RAG)	16
2.4 Real-Time Communication in Web Applications	16
2.5 Gap Analysis	16
3 Requirements Analysis	17
3.1 Functional Requirements	17
3.1.1 User Authentication Module (FR-1)	17
3.1.2 Professional Profile Module (FR-2)	17
3.1.3 Career Planning Module (FR-3)	17
3.1.4 AI Chatbot Module (FR-4)	18
3.1.5 Mentorship Module (FR-5)	18
3.1.6 Messaging Module (FR-6)	18
3.1.7 Video Meeting Module (FR-7)	19
3.1.8 Admin Module (FR-8)	19
3.2 Non-Functional Requirements	20
3.3 Use Case Diagram	21
3.4 User Stories	21
4 System Architecture and Design	23
4.1 High-Level Architecture	23
4.2 Technology Stack	24
4.2.1 Backend Technologies	24

4.2.2	Frontend Technologies	25
4.3	Database Design	26
4.3.1	Entity-Relationship Diagram	26
4.3.2	Database Schema	27
4.4	API Architecture	27
4.4.1	RESTful API Design	27
5	Component-Level Design	29
5.1	Design Classes and Problem Domain Mapping	29
5.2	Service Layer Architecture	29
5.3	Persistent Data Sources	31
5.4	Behavioral Design: AI Career Plan Generation	31
5.5	Deployment Architecture	32
6	Interface Design	33
6.1	User Interface Architecture	33
6.2	UI Events and State Transitions	34
6.3	Interface Screenshots	34
6.3.1	Landing Page	34
6.3.2	Authentication Pages	35
6.3.3	Dashboard	35
6.3.4	Career Wizard	35
6.3.5	AI-Generated Career Plan	36
6.3.6	AI Chatbot	36
6.3.7	Mentor Discovery	37
6.3.8	Real-Time Messaging	37
6.3.9	Video Session	38
6.3.10	Professional Profile	38
6.3.11	Mentor Dashboard	39
6.3.12	Admin Dashboard	39
6.3.13	Dark Mode	39
7	Implementation Details	40
7.1	Backend Implementation	40
7.1.1	Custom User Model	40
7.1.2	JWT Token Lifecycle	41
7.1.3	LLM Provider Factory Pattern	41
7.1.4	FAISS Vector Database	41
7.1.5	WebSocket Consumers	42
7.1.6	Agora Video Integration	42
7.2	Frontend Implementation	43
7.2.1	Application Architecture	43
7.2.2	Axios Interceptor for Token Refresh	43

7.2.3	Theme System	44
7.2.4	WebSocket Hook for Real-Time Features	44
8	Testing	46
8.1	Testing Approach and Strategy	46
8.1.1	Testing Framework	46
8.1.2	Testing Strategy	46
8.2	Item Pass/Fail Criteria	47
8.3	Test Results Summary	47
8.4	Detailed Test Cases	47
8.4.1	Authentication Tests (accounts – 47 tests)	47
8.4.2	Career Planning Tests (career – 26 tests)	48
8.4.3	Mentorship Tests (mentorship – 23 tests)	49
8.4.4	Profiles Tests (profiles – 20 tests)	49
8.4.5	Chatbot Tests (chatbot – 12 tests)	50
8.4.6	Messaging Tests (messaging – 10 tests)	50
8.5	Bugs Found and Fixed During Testing	51
8.6	Risks and Contingencies	51
9	Deployment and DevOps	52
9.1	Docker Compose Architecture	52
9.2	Deployment Steps	52
9.3	Environment Configuration	53
10	User Manual	54
10.1	System Requirements	54
10.2	Getting Started	54
10.2.1	Registration	54
10.2.2	Sign In	54
10.3	Dashboard	55
10.4	Career Wizard	55
10.5	AI Career Plan	55
10.6	AI Chatbot	56
10.7	Finding and Connecting with Mentors	56
10.8	Scheduling Sessions	56
10.9	Messaging	57
10.10	Becoming a Mentor	57
10.11	Theme Switching	57
11	Conclusion and Future Work	58
11.1	Summary of Achievements	58
11.2	Challenges Faced	58
11.3	Limitations	59

11.4 Future Work	59
11.5 Concluding Remarks	60
References	61
A API Endpoint Reference	63
A.1 Authentication Endpoints	63
A.2 Career Planning Endpoints	63
A.3 Mentorship Endpoints	64
A.4 Chatbot Endpoints	64
A.5 Messaging & WebSocket Endpoints	64
B Database Schema Details	66
B.1 Complete Table Definitions	66
C Project Directory Structure	68

List of Figures

3.1	Use Case Diagram – Career Talk	21
4.1	High-Level System Architecture	23
4.2	Entity-Relationship Diagram	26
5.1	Service Layer Architecture Pattern	30
5.2	Multi-Agent Career Plan Generation Pipeline	31
5.3	Career Profile State Machine	31
5.4	Docker Compose Deployment Diagram	32
6.1	Career Talk Landing Page with Three.js Hyperspeed Effect	34
6.2	Authentication Pages – Sign In (left) and Sign Up (right)	35
6.3	User Dashboard with Career Statistics and Quick Actions	35
6.4	6-Phase Career Wizard – Step 1: Basic Info (left) and Step 5: Goals (right)	35
6.5	AI-Generated Career Plan – Career Tracks, Skill Gaps, and Roadmap	36
6.6	AI Career Chatbot with RAG-powered Contextual Responses	36
6.7	Mentor Discovery Page with Search, Filters, and Connection Actions	37
6.8	Real-Time Messaging with Online Status and Conversation List	37
6.9	Agora-Powered Video Session with Screen Sharing and Chat	38
6.10	Professional Profile – Skills, Experience, Education, Achievements	38
6.11	Mentor Dashboard – Availability, Sessions, Statistics, Connections	39
6.12	Admin Dashboard – Platform Statistics and User Management	39
6.13	Theme Support – Light Mode (left) and Dark Mode (right)	39

List of Tables

2.1	Comparison of Existing Career Guidance Platforms	15
3.1	Non-Functional Requirements	20
4.1	Backend Technology Stack	24
4.2	Frontend Technology Stack	25
4.3	Core Database Tables	27
4.4	API Endpoint Summary	28
5.1	Problem Domain to Design Class Mapping	29
5.2	Persistent Data Sources	31
6.1	UI Objects and Actions	33
6.2	Key UI Events and State Changes	34
8.1	Overall Test Execution Results	47
8.2	Test Distribution by Application	47
8.5	Bugs Discovered and Resolved During Testing	51
8.6	Testing Risks and Mitigation Strategies	51
9.1	Key Environment Variables	53
10.1	Minimum System Requirements	54

List of Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DB	Database
DRF	Django REST Framework
FAISS	Facebook AI Similarity Search
HNSW	Hierarchical Navigable Small World
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JWT	JSON Web Token
LLM	Large Language Model
ORM	Object-Relational Mapping
OTP	One-Time Password
PK	Primary Key
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
RTC	Real-Time Communication
SDK	Software Development Kit
SPA	Single Page Application
SQL	Structured Query Language
SRS	Software Requirements Specification
UI	User Interface
UUID	Universally Unique Identifier
UX	User Experience
WS	WebSocket
WSGI	Web Server Gateway Interface

Chapter 1

Introduction

1.1 Background and Motivation

The global career guidance landscape faces a notable accessibility crisis. In South Asia, quality career counselling tends to be available mainly through expensive consultants, select universities, or personal networks, leaving millions of students and early-career professionals without proper guidance at critical decision-making points in their lives.

A 2023 UNESCO report found that over 60% of university graduates in developing countries feel inadequately prepared for the job market, while fewer than 15% have had access to professional career counselling services. This gap results in mismatched career choices, prolonged unemployment, and underutilization of talent.

With the rise of Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG) technology, there is now an unprecedented opportunity to make career guidance more widely available. By pairing AI-generated personalized advice with a human mentorship network and interactive communication tools, it becomes possible to build a platform that is accessible, affordable, and scalable.

1.2 Problem Statement

Students and early-career professionals in South Asia run into several interconnected challenges:

1. **Limited Access to Career Guidance:** Professional career counselling services are geographically concentrated in urban centres and financially prohibitive for most.
2. **Generic Advice:** Available career advice is often one-size-fits-all, failing to account for individual backgrounds, skills, and aspirations.
3. **Mentor Discovery Gap:** Finding experienced industry mentors is largely dependent on personal networks, creating inequality.
4. **Information Overload:** The abundance of unstructured career information online leads to analysis paralysis rather than clarity.
5. **Lack of Structured Planning:** Most individuals lack a systematic approach to skill gap analysis and career roadmap creation.

1.3 Proposed Solution

Career Talk tackles these challenges through a unified platform organized around three main areas:

1. **AI-Powered Career Planning:** A 6-phase career profiling wizard feeds into a multi-agent LLM pipeline that produces personalized career tracks, skill gap analysis, professional roadmaps, and resource recommendations.
2. **Agentic AI Chatbot:** A context-aware conversational AI assistant that draws on the user's profile data and a RAG-powered knowledge base to deliver interactive, domain-specific career guidance.
3. **Human Mentorship Network:** A discovery-and-connection system where users can find industry mentors, schedule sessions, communicate via real-time messaging, and hold video meetings.

1.4 Objectives

The main objectives behind the Career Talk project are as follows:

- Design and build a full-stack web application that follows modern software engineering practices.
- Develop an AI career planning engine using LangChain, Google Gemini, and a FAISS vector database.
- Build a real-time mentorship system featuring WebSocket messaging and Agora video conferencing.
- Create a production-ready REST API architecture with JWT authentication and role-based access control.
- Deliver a responsive, accessible UI with dark/light theme support.
- Document the entire software development lifecycle from requirements through deployment.

1.5 Scope

The scope of Career Talk covers the following areas:

- User registration with email/password and Google OAuth authentication.

- Professional profile management with skills, education, work experience, and achievements.
- 6-phase career profiling wizard with schema-driven dynamic forms.
- AI-generated career plans with multiple career tracks, skill gap analysis, and roadmaps.
- AI chatbot with RAG-based contextual responses.
- Mentor profile management and discovery with search and filtering.
- Connection request workflow between mentees and mentors.
- Session scheduling with video/audio/chat options.
- Real-time messaging with online status indicators and unread badges.
- Video meetings powered by Agora RTC SDK.
- Admin dashboard for platform management.
- Responsive design optimized for desktop and mobile devices.

1.6 Report Organization

This report is organized as follows:

- **Chapter 2:** Literature Review and Related Work
- **Chapter 3:** Requirements Analysis (SRS)
- **Chapter 4:** System Architecture and Design
- **Chapter 5:** Component-Level Design
- **Chapter 6:** Interface Design
- **Chapter 7:** Implementation Details
- **Chapter 8:** Testing
- **Chapter 9:** Deployment and DevOps
- **Chapter 10:** User Manual
- **Chapter 11:** Conclusion and Future Work

Chapter 2

Literature Review and Related Work

2.1 Existing Career Guidance Platforms

A number of platforms currently offer career guidance services. Each one takes a different approach, and each has its own set of limitations:

Table 2.1: Comparison of Existing Career Guidance Platforms

Platform	Strengths	Limitations
LinkedIn	Vast professional network, job recommendations	Generic suggestions, no personalized AI career plans
MentorCruise	Quality mentor matching, structured sessions	Expensive, limited AI integration
Roadmap.sh	Excellent visual career roadmaps	Static content, no personalization, no mentorship
ChatGPT	Conversational AI, broad knowledge	No career-specific RAG, no user profiles, no mentorship
Handshake	University-focused job matching	Limited to enrolled students, US-centric

2.2 Large Language Models in Career Guidance

The rapid evolution of LLMs, spanning from GPT-3 to GPT-4, Gemini, and open-source models like LLaMA, has opened new avenues for AI-powered career advice. Some of the key research areas in this space include:

- **Structured Output Generation:** Using LLMs to generate structured career plans (JSON schemas) rather than free-text responses, ensuring programmatic usability.
- **Multi-Agent Systems:** Orchestrating multiple specialized AI agents (profile analysis, plan generation, evaluation, skill analysis) to produce higher-quality outputs than a single prompt.
- **Domain-Specific Fine-Tuning:** Augmenting general-purpose LLMs with career domain knowledge through RAG and prompt engineering.

2.3 Retrieval-Augmented Generation (RAG)

RAG brings together the generative capabilities of LLMs with the factual accuracy of information retrieval systems. In the context of Career Talk, the relevant components are:

- **FAISS:** Facebook AI Similarity Search provides efficient nearest-neighbour lookup over dense vector embeddings, used for storing and querying career knowledge documents.
- **Sentence Transformers:** The `all-MiniLM-L6-v2` model generates 384-dimensional embeddings for career documents and user queries, enabling semantic similarity search.
- **Hybrid Search:** Combining vector similarity with metadata filtering to retrieve contextually relevant career information.

2.4 Real-Time Communication in Web Applications

Any modern mentorship platform needs real-time communication capabilities. The key technologies used are:

- **WebSocket Protocol:** Enables full-duplex communication between client and server, used for instant messaging and live notifications.
- **Django Channels:** Extends Django to handle WebSocket connections via ASGI, enabling real-time features within the Django ecosystem.
- **WebRTC & Agora:** Agora's RTC SDK provides enterprise-grade video/audio communication with minimal latency, handling NAT traversal, codec management, and network adaptation.

2.5 Gap Analysis

Career Talk fills a distinct niche by bringing together:

1. Personalized AI career planning (unlike static platforms like Roadmap.sh)
2. RAG-powered contextual guidance (unlike generic ChatGPT)
3. Integrated human mentorship (unlike AI-only solutions)
4. Real-time communication (unlike asynchronous email-based platforms)
5. Accessibility focus for South Asian users (unlike US/Europe-centric platforms)

Chapter 3

Requirements Analysis

3.1 Functional Requirements

3.1.1 User Authentication Module (FR-1)

- **FR-1.1:** Users shall register using email and password with OTP verification.
- **FR-1.2:** Users shall authenticate via Google OAuth 2.0 (authorization code flow).
- **FR-1.3:** JWT access tokens (15-minute lifetime) and refresh tokens (30-day lifetime, HttpOnly cookies) shall manage sessions.
- **FR-1.4:** Progressive account lockout after 5 failed login attempts (30-minute lock).
- **FR-1.5:** Password management: forgot password via OTP, reset, and change flows.
- **FR-1.6:** Role-based access control with User and Admin roles.

3.1.2 Professional Profile Module (FR-2)

- **FR-2.1:** Users shall create and manage professional profiles with title, company, location, bio.
- **FR-2.2:** Users shall add skills with proficiency levels, categories, and percentages.
- **FR-2.3:** Users shall add work experiences, education entries, achievements, and certifications.
- **FR-2.4:** Profile completeness shall be automatically calculated.
- **FR-2.5:** AI-identified top strengths and improvement areas shall be displayed.

3.1.3 Career Planning Module (FR-3)

- **FR-3.1:** A 6-phase career profiling wizard shall collect user data (basic info, education, skills, experience, goals, preferences).
- **FR-3.2:** Schema-driven dynamic forms shall adapt based on user inputs.

- **FR-3.3:** Auto-save and session restoration for wizard progress.
- **FR-3.4:** AI career plan generation using multi-agent LangChain pipeline.
- **FR-3.5:** Generated plans shall include career tracks, skill gaps, roadmap, resources, and opportunities.
- **FR-3.6:** Plan refinement based on user feedback.
- **FR-3.7:** Plan evaluation and benchmarking by AI evaluation agent.

3.1.4 AI Chatbot Module (FR-4)

- **FR-4.1:** Users shall create and manage multiple chat sessions.
- **FR-4.2:** The chatbot shall provide context-aware career guidance using RAG.
- **FR-4.3:** Chat history shall be persisted and browsable.
- **FR-4.4:** The chatbot shall use the user's profile data for personalized responses.
- **FR-4.5:** Support for multiple session types (AI chat, career guidance, skill advice).

3.1.5 Mentorship Module (FR-5)

- **FR-5.1:** Users shall create mentor profiles with expertise, industries, availability, and rates.
- **FR-5.2:** Mentor discovery with search, filter (expertise, industry, rating, price), and sort.
- **FR-5.3:** Connection request workflow: send, accept, reject, cancel.
- **FR-5.4:** Session scheduling with video, audio, and chat options.
- **FR-5.5:** Session lifecycle: scheduled -> in-progress -> completed/cancelled/no-show.
- **FR-5.6:** Post-session feedback and rating system (1–5 scale).
- **FR-5.7:** Mentor dashboard with statistics, session management, and availability configuration.

3.1.6 Messaging Module (FR-6)

- **FR-6.1:** Real-time messaging via WebSocket (Django Channels).
- **FR-6.2:** Direct and mentorship conversation types.

- **FR-6.3:** Online status indicators (real-time via NotificationConsumer).
- **FR-6.4:** Unread message count with live updates in navigation.
- **FR-6.5:** Message read receipts and mark-as-read functionality.
- **FR-6.6:** System messages for automated notifications.

3.1.7 Video Meeting Module (FR-7)

- **FR-7.1:** Agora RTC-based video and audio calls.
- **FR-7.2:** Screen sharing capability.
- **FR-7.3:** Server-side token generation for secure RTC sessions.
- **FR-7.4:** Session duration tracking with actual start/end recording.

3.1.8 Admin Module (FR-8)

- **FR-8.1:** Admin dashboard with platform statistics (total users, sessions, plans).
- **FR-8.2:** User management: list, search, activate/deactivate.
- **FR-8.3:** Role-based access restricting admin features to admin users only.

3.2 Non-Functional Requirements

Table 3.1: Non-Functional Requirements

ID	Category	Requirement
NFR-1	Performance	API response time < 500ms for standard operations; LLM calls < 60s
NFR-2	Scalability	Horizontal scaling via Docker Compose; Redis-backed caching and task queues
NFR-3	Security	JWT with rotation and blacklisting; SHA-256 hashed OTPs; Argon2 passwords; CORS restriction in production
NFR-4	Availability	Health check endpoint at <code>/api/v1/health/</code> ; Docker health checks; graceful Redis fallback
NFR-5	Usability	Responsive design (320px – 1440px+); Dark/Light themes; WCAG-aligned accessibility
NFR-6	Maintainability	Service layer pattern; modular Django apps; centralized configuration via <code>.env</code>
NFR-7	Reliability	Celery task retry logic; LLM fallback chain (primary -> fallback -> mock); database atomic transactions
NFR-8	Compatibility	Chrome, Firefox, Safari, Edge (latest 2 versions); iOS Safari, Android Chrome

3.3 Use Case Diagram

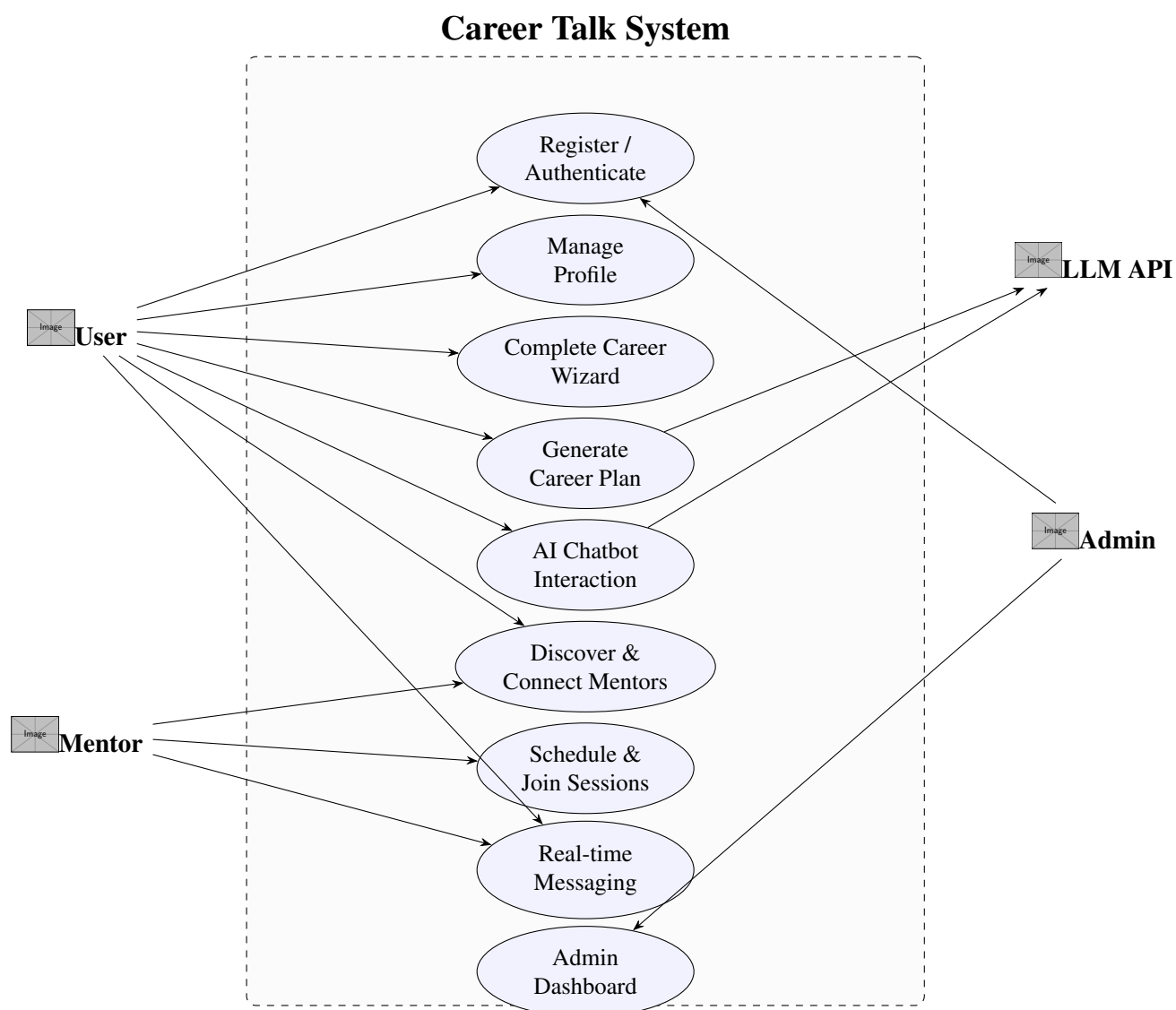


Figure 3.1: Use Case Diagram – Career Talk

3.4 User Stories

ID	Role	User Story
US-01	Student	As a student, I want to create an account so that I can access personalized career guidance.
US-02	User	As a user, I want to complete a career wizard so that the AI understands my background and goals.
US-03	User	As a user, I want to receive an AI-generated career plan so that I have a clear professional roadmap.

ID	Role	User Story
US-04	User	As a user, I want to chat with an AI assistant so that I can get instant answers to career questions.
US-05	User	As a user, I want to search for mentors so that I can find experienced professionals in my field.
US-06	User	As a user, I want to schedule video sessions with mentors so that I can receive personalized guidance.
US-07	Mentor	As a mentor, I want to manage my availability so that mentees can book sessions at convenient times.
US-08	Mentor	As a mentor, I want to view my session statistics so that I can track my mentoring impact.
US-09	User	As a user, I want real-time messaging so that I can communicate with mentors asynchronously.
US-10	Admin	As an admin, I want to manage users so that I can maintain platform integrity.

Chapter 4

System Architecture and Design

4.1 High-Level Architecture

Career Talk is structured as a **three-tier client-server application** with a clear separation between the presentation layer (React SPA), the business logic layer (Django REST API), and the data layer (PostgreSQL + Redis + FAISS).

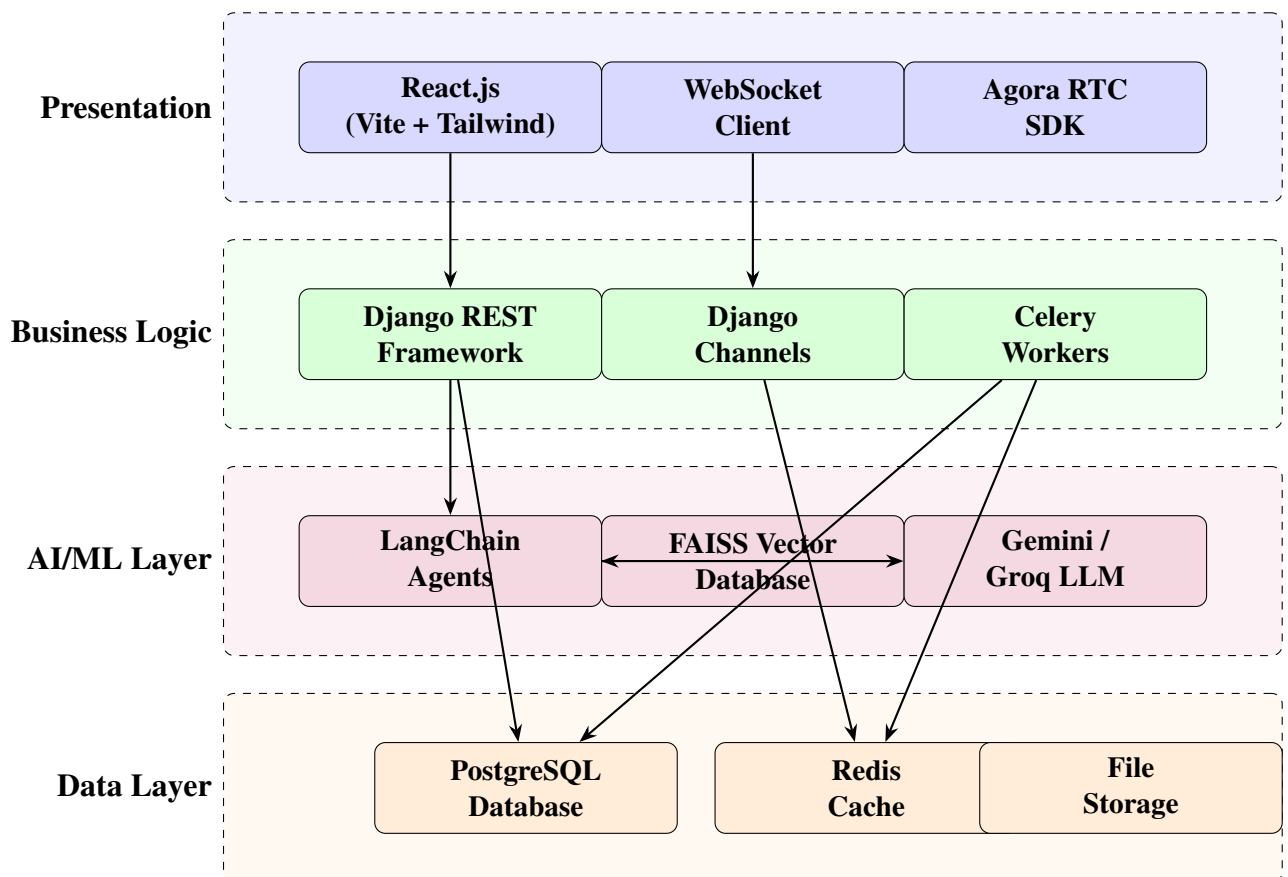


Figure 4.1: High-Level System Architecture

4.2 Technology Stack

4.2.1 Backend Technologies

Table 4.1: Backend Technology Stack

Component	Technology	Purpose
Framework	Django 5.0.2	Python web framework, ORM, admin
API Layer	DRF 3.14	RESTful API endpoints, serialization, permissions
Authentication	SimpleJWT 5.3.1	JWT token generation, validation, blacklisting
OAuth	Custom GoogleOAuth	Google sign-in via authorization code flow
Task Queue	Celery 5.3.4	Async task processing (email, plan generation)
Message Broker	Redis 5.0.1	Celery broker, cache, WebSocket channel layer
Database	PostgreSQL 14+	Primary relational database with UUID PKs
Vector DB	FAISS (faiss-cpu)	HNSW index for semantic search (384-dim)
LLM Framework	LangChain 0.3.27	Multi-agent AI pipeline orchestration
LLM Providers	Gemini / Groq	Primary and fallback LLM APIs
Embeddings	sentence-transformers	all-MiniLM-L6-v2 (384 dims, free)
WebSocket	Django Channels 4.0	ASGI, real-time messaging, notifications
Video	Agora RTC	Video/audio meetings with token auth
API Docs	drf-yasg 1.21.11	Swagger UI and ReDoc auto-generated docs
Containerization	Docker Compose	Multi-service deployment orchestration

4.2.2 Frontend Technologies

Table 4.2: Frontend Technology Stack

Component	Technology	Purpose
Framework	React 19.1.1	Component-based UI (JavaScript, not TypeScript)
Build Tool	Vite 7.1.7	Fast dev server, HMR, optimized production builds
Styling	Tailwind CSS 4.1	Utility-first CSS with dark/light theme support
Components	shadcn/ui	Accessible component primitives (buttons, inputs)
Animation	Motion (Framer) 12.23	Page transitions and element animations
3D Effects	Three.js 0.180	Hyperspeed particle effect on landing page
HTTP Client	Axios 1.12.2	API calls with JWT interceptor and token refresh
Routing	React Router 7.9	Client-side routing with role-based guards
State Management	React Context	AuthContext, CareerContext, ThemeContext, ChatContext
OAuth	@react-oauth/google	Google sign-in popup (auth-code flow)
Video	agora-rtc-sdk-ng	WebRTC video/audio in-browser
Notifications	react-hot-toast	Toast notifications for success/error feedback

4.3 Database Design

4.3.1 Entity-Relationship Diagram

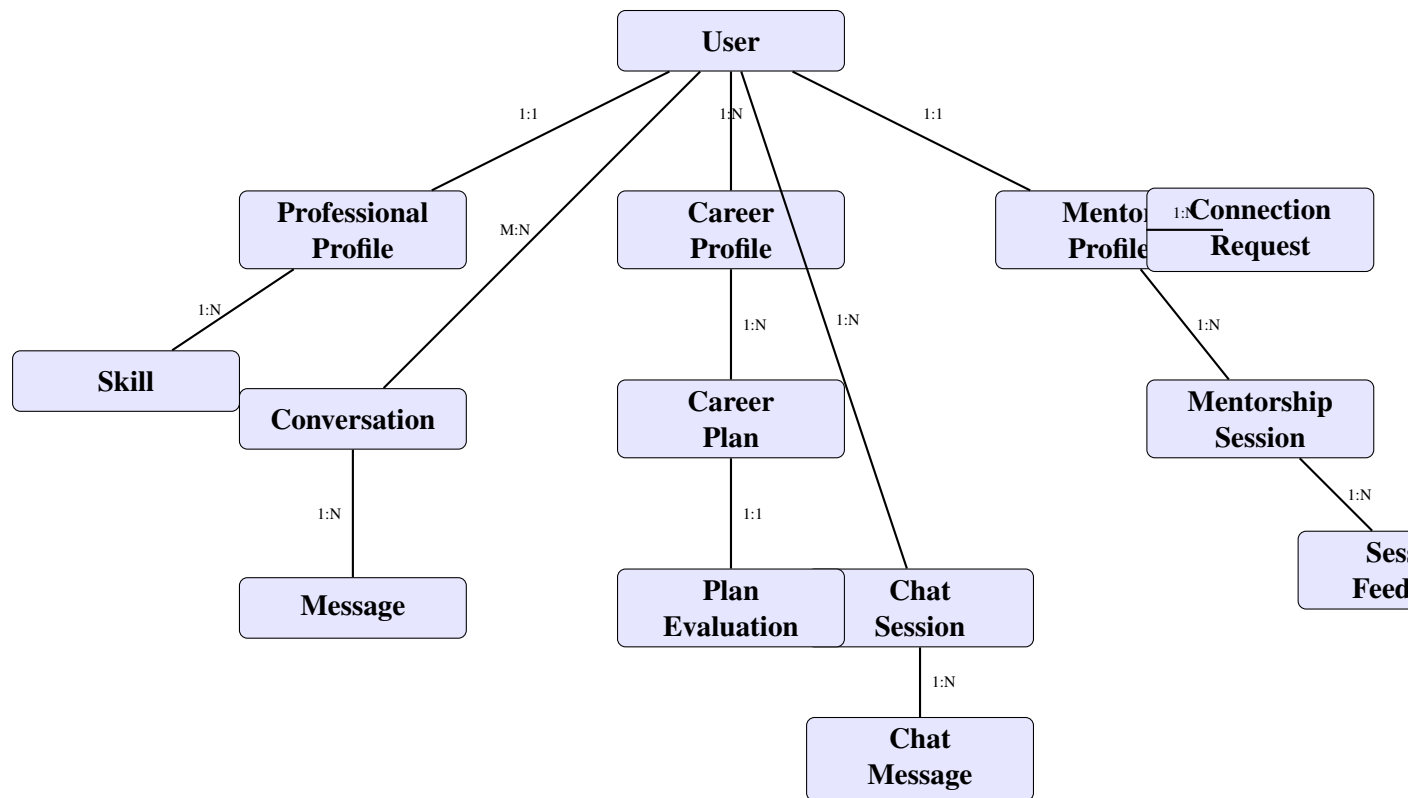


Figure 4.2: Entity-Relationship Diagram

4.3.2 Database Schema

Table 4.3: Core Database Tables

Table	App	Key Fields
users	accounts	id (UUID PK), email, full_name, role, google_id, is_verified, locked_until
refresh_tokens	accounts	token, user_id (FK), expires_at, is_revoked, user_agent, ip_address
otps	accounts	hashed_otp (SHA-256), user_id (FK), purpose, expires_at, attempts
professional_profiles	profiles	user_id (1:1), title, company, location, bio, career_score, completeness
skills	profiles	profile_id (FK), name, proficiency, proficiency_percentage, category
work_experiences	profiles	profile_id (FK), job_title, company_name, start_date, end_date
education	profiles	profile_id (FK), degree, institution, field_of_study, grade
career_profiles	career	user_id (FK), phase_1..6 (JSON), status, current_step
career_plans	career	profile_id (FK), career_tracks (JSON), skill_gaps, roadmap, summary
plan_evaluations	career	plan_id (1:1), alignment/consistency/relevance/completeness/actionability scores
chat_sessions	chatbot	user_id (FK), title, session_type, message_count
chat_messages	chatbot	session_id (FK), role, content, tokens_used, processing_time
mentor_profiles	mentorship	user_id (1:1), headline, expertise_areas, average_rating, total_sessions
mentorship_sessions	mentorship	mentor_id, mentee_id, scheduled_at, status, duration_minutes
conversations	messaging	participants (M2M), conversation_type, is_active
messages	messaging	conversation_id (FK), sender_id (FK, nullable), content, is_read

4.4 API Architecture

4.4.1 RESTful API Design

Every API endpoint follows the `/api/v1/{module}/` convention and uses a consistent response format:

Table 4.4: API Endpoint Summary

Module	Endpoints	Base Path
Authentication	12	/api/v1/auth/
Professional Profile	14	/api/v1/profiles/
Career Planning	10	/api/v1/career/
Chatbot	4	/api/v1/chatbot/
Mentorship	14	/api/v1/mentorship/
Messaging (REST)	5	/api/v1/messaging/
Messaging (WebSocket)	2	ws://{host}/ws/
Admin	5	/api/v1/admin/
Health	1	/api/v1/health/
Total	67	

Chapter 5

Component-Level Design

5.1 Design Classes and Problem Domain Mapping

Career Talk is organized into seven Django applications, each covering a distinct problem domain. The table below shows how problem domain concepts map to their design classes:

Table 5.1: Problem Domain to Design Class Mapping

Domain Concept	Design Class	Responsibilities
User Identity	User model	Email-based auth, roles, logout, OAuth
Token Management	TokenService	JWT generation, validation, refresh, blacklist
OTP Verification	OTP model + AuthService	SHA-256 hashed OTP create/verify/expire
Professional Profile	ProfessionalProfile + related	Skills, experience, education management
Career Profiling	CareerProfile	6-phase JSON data collection
AI Plan Generation	CareerAgent	Multi-agent LLM pipeline orchestration
Career Plan	CareerPlan	Structured plan storage (tracks, gaps, roadmap)
Plan Quality	PlanEvaluation	AI-generated quality scores (5 dimensions)
Chat Session	ChatSession + ChatMessage	AI conversation management
Mentor Profile	MentorProfile	Expertise, availability, ratings
Connection	ConnectionRequest	Mentee-to-mentor connection workflow
Session	MentorshipSession	Scheduled meeting lifecycle
Conversation	Conversation + Message	Real-time messaging data
Online Status	UserOnlineStatus	WebSocket-tracked presence

5.2 Service Layer Architecture

The backend uses a **three-layer architecture** pattern that keeps concerns separate:

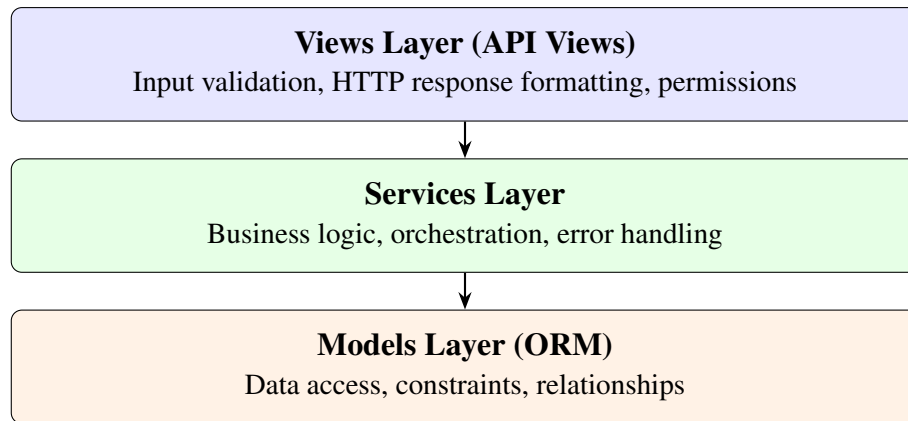


Figure 5.1: Service Layer Architecture Pattern

The main service classes are:

- **AuthService:** Handles signup, signin, OTP verification, logout logic
- **TokenService:** JWT creation, refresh rotation, Redis blacklisting
- **EmailService:** Provider-agnostic email dispatch (Console/SMTP/SendGrid)
- **CareerService:** Career wizard logic, profile creation, step submission
- **CareerAgent:** Multi-agent LLM pipeline for career plan generation
- **LLMProvider:** Factory pattern for LLM instantiation (Gemini/Groq/Mock)
- **MentorService:** Mentor discovery, connection management, session lifecycle
- **VideoService:** Agora RTC token generation, session tracking
- **EmbeddingService:** Sentence-transformer embedding generation
- **FAISSVectorDB:** Thread-safe vector database operations
- **CareerKnowledgeBase:** Domain-specific FAISS facade

5.3 Persistent Data Sources

Table 5.2: Persistent Data Sources

Data Source	Type	Description
PostgreSQL	Relational DB	Primary data store that holds users, profiles, plans, sessions, messages (30+ tables across 7 apps)
Redis	In-memory KV	JWT blacklist, session cache, Celery broker, Channels layer, rate-limit counters
FAISS Index	Binary Files	<code>career_knowledge.index</code> and <code>.metadata</code> – HNSW vector index with career knowledge embeddings
File System	Log Files	<code>logs/django.log</code> – Rotating log files (10MB, 5 backups)

5.4 Behavioral Design: AI Career Plan Generation

The career plan generation follows a multi-agent pipeline pattern:

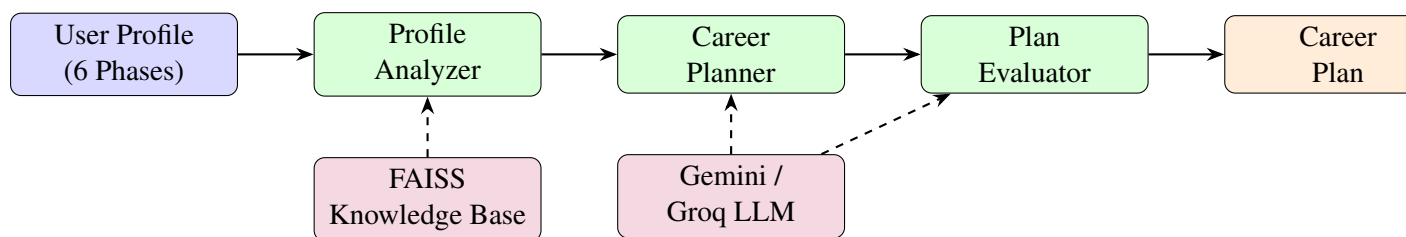


Figure 5.2: Multi-Agent Career Plan Generation Pipeline

State Diagram – Career Profile Lifecycle:

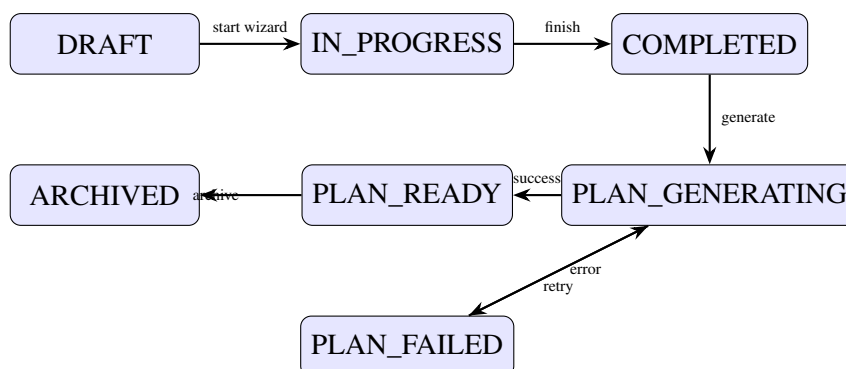


Figure 5.3: Career Profile State Machine

5.5 Deployment Architecture

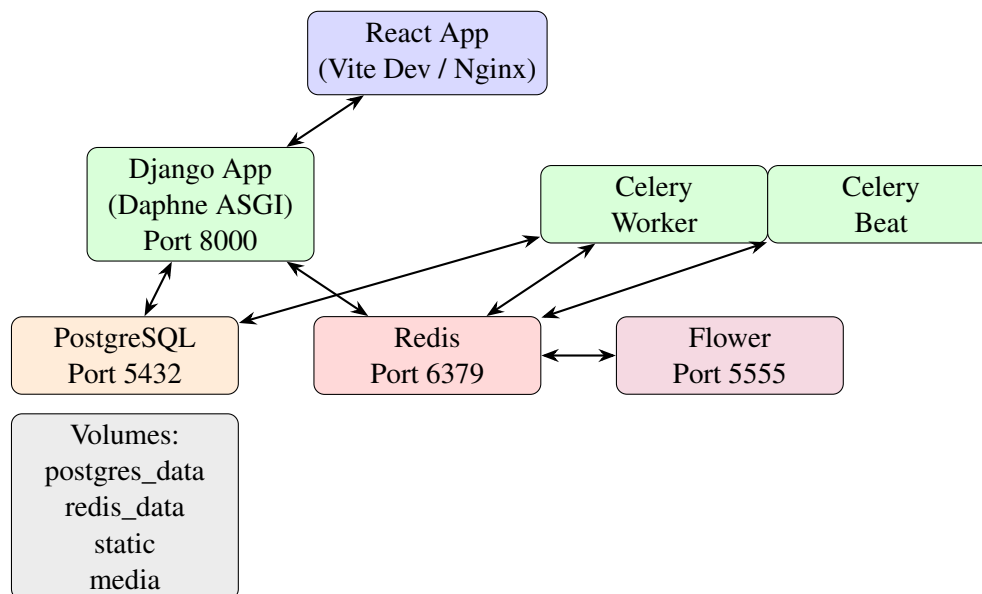


Figure 5.4: Docker Compose Deployment Diagram

Chapter 6

Interface Design

6.1 User Interface Architecture

The frontend uses a **component-based architecture** with React Context handling global state management. Below are the key UI objects and their hierarchical relationships:

Table 6.1: UI Objects and Actions

UI Object	Type	Actions (Operations)
FloatingNavbar	Navigation	Navigate routes, toggle mobile menu, show unread badge
AuthForms	Form Group	Submit signup/signin, Google OAuth popup, OTP input
CareerWizard	Multi-step Form	Navigate steps (1–6), auto-save, submit, restore session
PlanView	Data Display	View career tracks, skill gaps, roadmap; refine plan
ChatWidget	Floating Widget	Open/close chat, send messages, view history, create sessions
ExpertsGrid	Discovery	Search mentors, apply filters, view profiles, send connections
SessionDetail	Management	View details, add notes, join video, give feedback, cancel
MessagingPanel	Communication	Select conversation, send/receive messages, view online status
ProfileEditor	CRUD Form	Edit profile fields, manage skills/experience/education
AdminDashboard	Dashboard	View stats, manage users, toggle activation
ThemeToggle	Control	Switch between dark and light themes

6.2 UI Events and State Transitions

Table 6.2: Key UI Events and State Changes

User Event	Action	UI State Change
Click “Sign In”	POST /auth/signin	Loading -> Dashboard redirect / Error toast
Complete Wizard Step	PATCH /career/{id}/step/	Progress bar advances, next step renders
Click “Generate Plan”	POST /career/{id}/finish/	Polling state -> Plan view when ready
Send Chat Message	POST /chatbot/messages/send	Message appears, AI response streams in
Click “Connect” (Mentor)	POST /mentorship/connection	Button state: Connect -> Pending
Send Message (Chat)	WebSocket send_message	Message appends to conversation in real-time
Toggle Theme	LocalStorage update	CSS vars switch, all components re-render
Join Video Session	Agora client join channel	Video/audio streams render in session page

6.3 Interface Screenshots

The figures below show the actual application interface. Each screenshot demonstrates a different part of the platform.

6.3.1 Landing Page

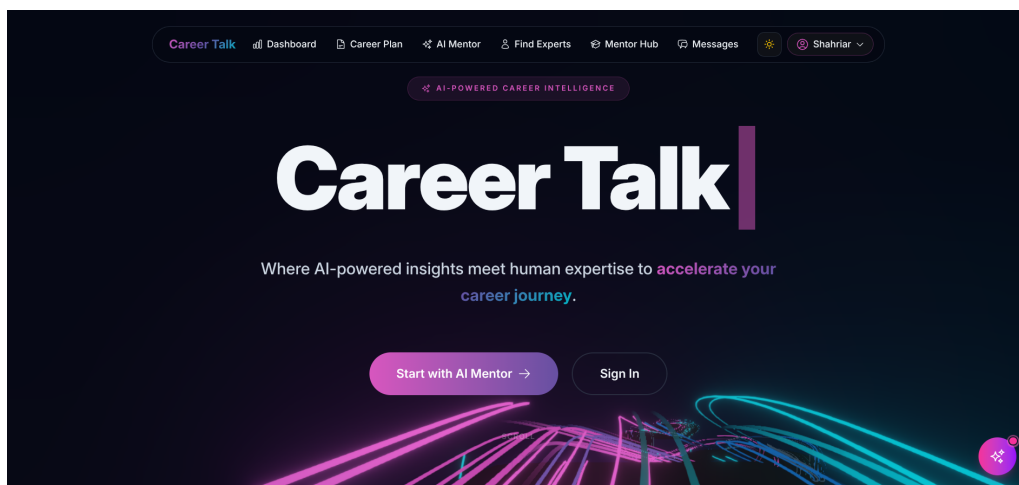


Figure 6.1: Career Talk Landing Page with Three.js Hyperspeed Effect

6.3.2 Authentication Pages



Figure 6.2: Authentication Pages – Sign In (left) and Sign Up (right)

6.3.3 Dashboard

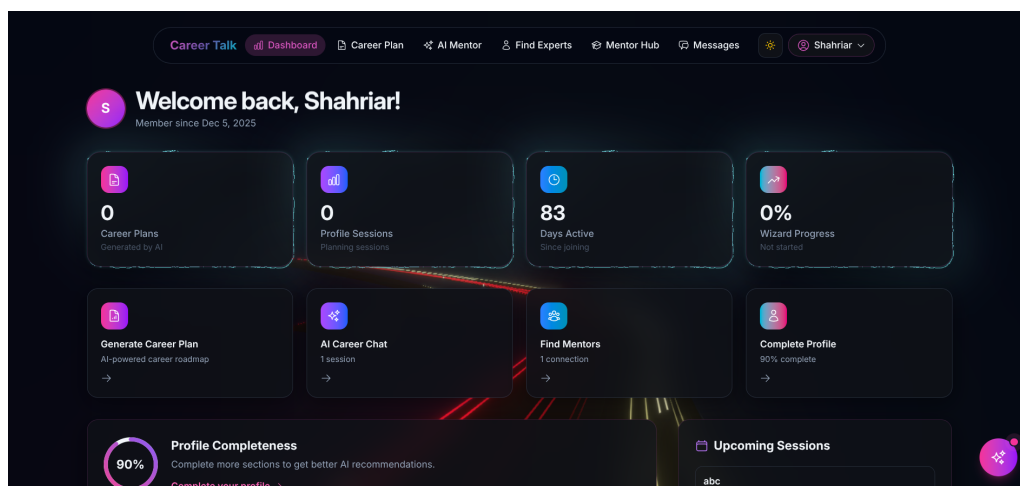


Figure 6.3: User Dashboard with Career Statistics and Quick Actions

6.3.4 Career Wizard



Figure 6.4: 6-Phase Career Wizard – Step 1: Basic Info (left) and Step 5: Goals (right)

6.3.5 AI-Generated Career Plan

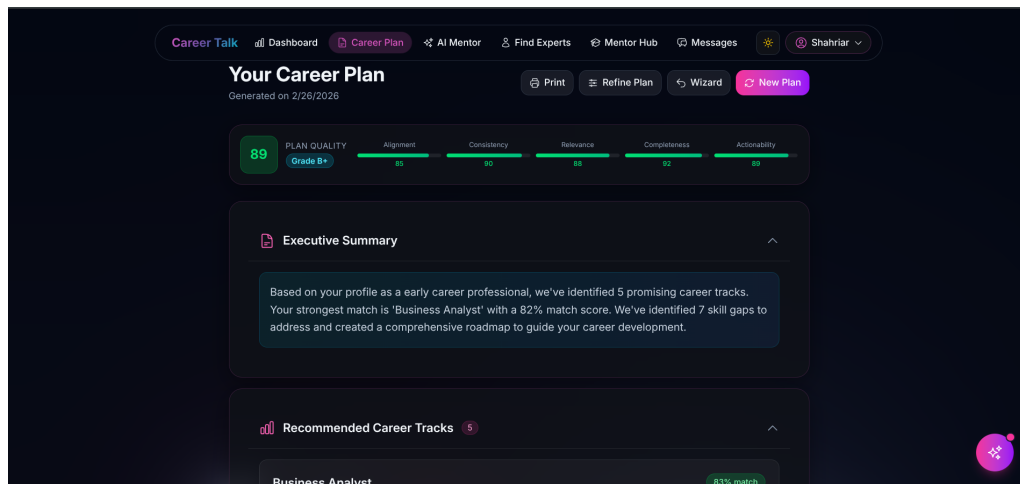


Figure 6.5: AI-Generated Career Plan – Career Tracks, Skill Gaps, and Roadmap

6.3.6 AI Chatbot

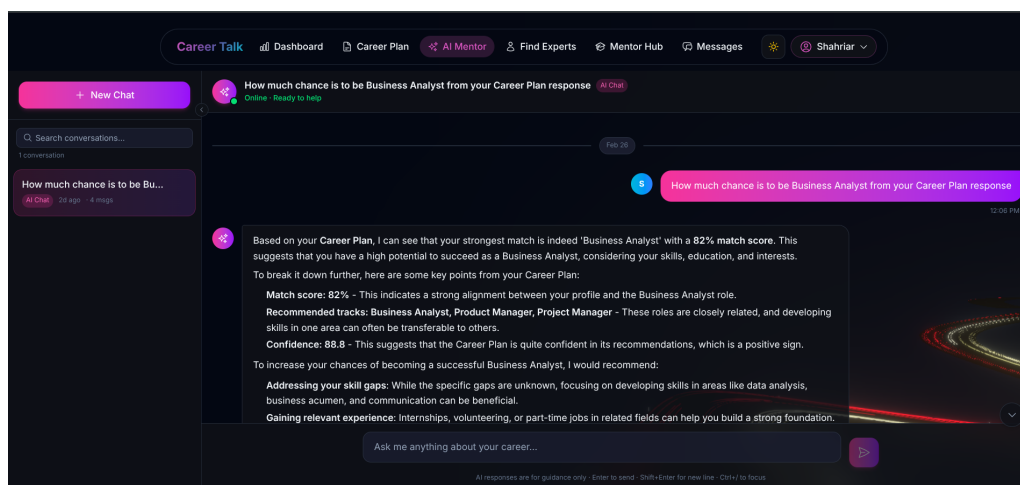


Figure 6.6: AI Career Chatbot with RAG-powered Contextual Responses

6.3.7 Mentor Discovery

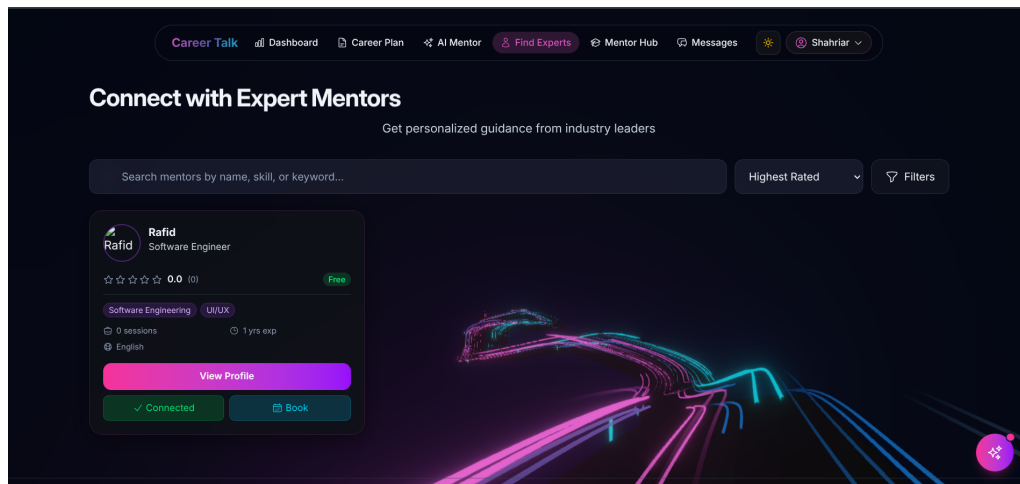


Figure 6.7: Mentor Discovery Page with Search, Filters, and Connection Actions

6.3.8 Real-Time Messaging

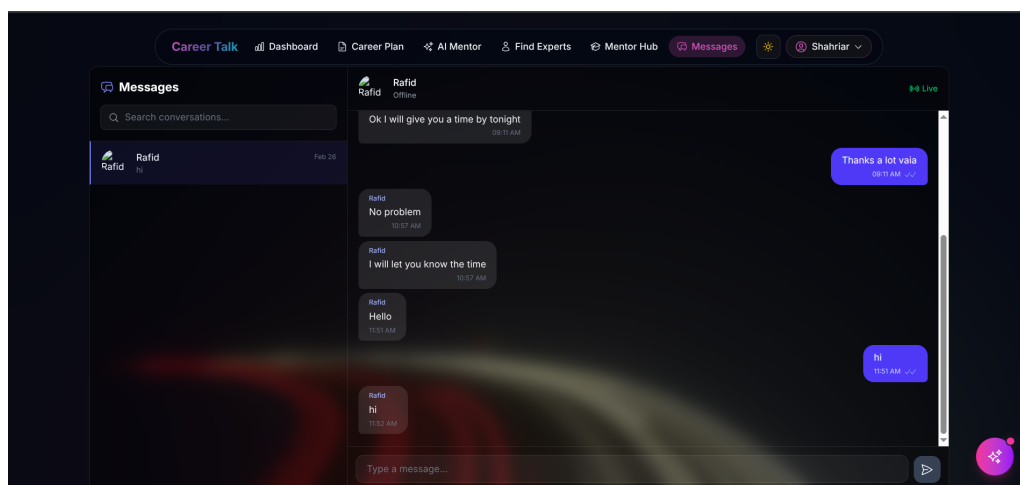


Figure 6.8: Real-Time Messaging with Online Status and Conversation List

6.3.9 Video Session

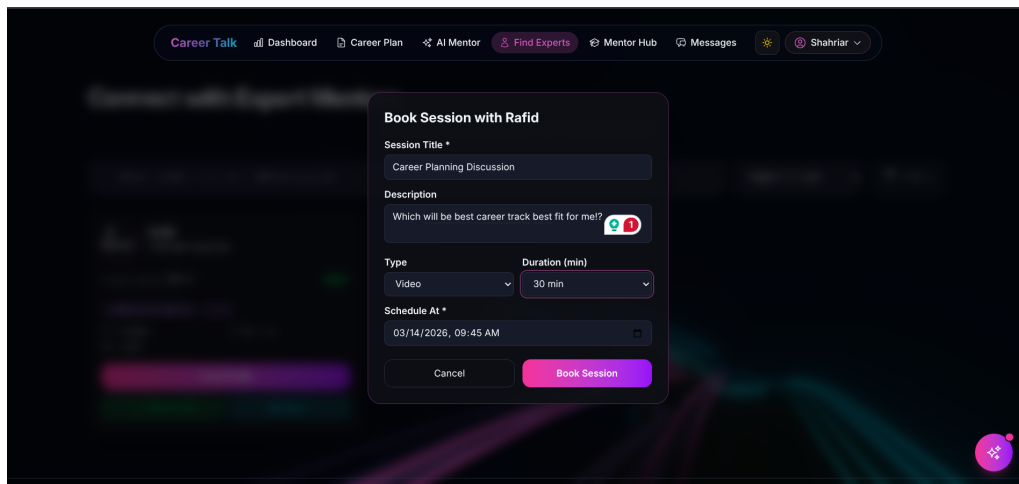


Figure 6.9: Agora-Powered Video Session with Screen Sharing and Chat

6.3.10 Professional Profile

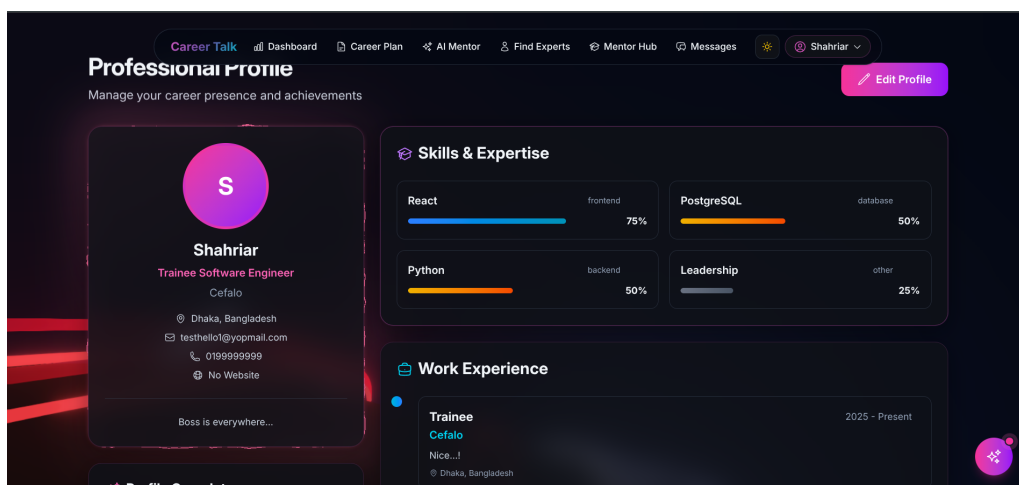


Figure 6.10: Professional Profile – Skills, Experience, Education, Achievements

6.3.11 Mentor Dashboard

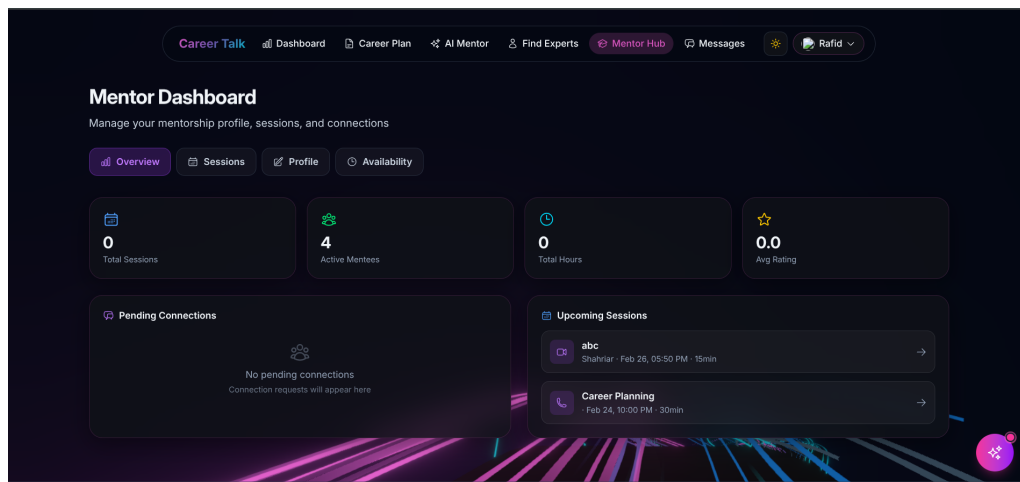


Figure 6.11: Mentor Dashboard – Availability, Sessions, Statistics, Connections

6.3.12 Admin Dashboard

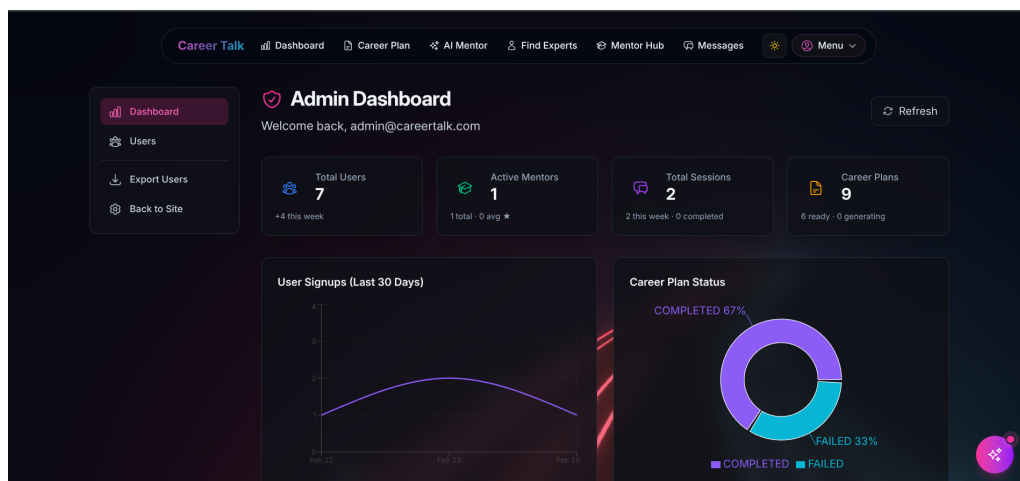


Figure 6.12: Admin Dashboard – Platform Statistics and User Management

6.3.13 Dark Mode



Figure 6.13: Theme Support – Light Mode (left) and Dark Mode (right)

Chapter 7

Implementation Details

7.1 Backend Implementation

7.1.1 Custom User Model

The authentication system relies on a custom User model that extends Django's `AbstractBaseUser` and `PermissionsMixin`:

```
1 class User(AbstractBaseUser, PermissionsMixin):
2     id = models.UUIDField(primary_key=True, default=uuid.uuid4,
3                             editable=False)
4     email = models.EmailField(max_length=320, unique=True,
5                                db_index=True)
6     full_name = models.CharField(max_length=100, blank=True, null=
7                                    True)
8     role = models.CharField(max_length=20, choices=UserRole.
9                               choices,
10                                default=UserRole.USER)
11     google_id = models.CharField(max_length=100, unique=True, null=
12                                     =True)
13     failed_login_attempts = models.IntegerField(default=0)
14     locked_until = models.DateTimeField(blank=True, null=True)
15
16     def is_locked(self):
17         if self.locked_until is None:
18             return False
19         return timezone.now() < self.locked_until
20
21     def lock_account(self, duration_minutes=None):
22         if duration_minutes is None:
23             duration_minutes = settings.LOCKOUT_DURATION_MINUTES
24         self.locked_until = timezone.now() + timedelta(minutes=
25                                                         duration_minutes)
26         self.save(update_fields=['locked_until'])
```

Listing 7.1: Custom User Model (accounts/models.py)

7.1.2 JWT Token Lifecycle

Token management follows a dual-storage approach. Access tokens are short-lived (15 minutes) and kept in localStorage, while refresh tokens are long-lived (30 days) and stored in HttpOnly cookies. Refresh tokens are rotated on each use.

```
1 SIMPLE_JWT = {
2     'ACCESS_TOKEN_LIFETIME': timedelta(minutes=15),
3     'REFRESH_TOKEN_LIFETIME': timedelta(days=30),
4     'ROTATE_REFRESH_TOKENS': True,
5     'BLACKLIST_AFTER_ROTATION': True,
6     'AUTH_HEADER_TYPES': ('Bearer',),
7     'USER_ID_FIELD': 'id',
8     'USER_ID_CLAIM': 'user_id',
9 }
```

Listing 7.2: JWT Configuration (settings.py)

7.1.3 LLM Provider Factory Pattern

The LLM provider is built on a factory pattern that supports multiple providers and includes automatic fallback:

```
1 def get_llm(provider=None, model=None, temperature=None):
2     provider = provider or settings.LLM_PROVIDER # gemini, groq,
3     mock
4     if provider == 'gemini':
5         return ChatGoogleGenerativeAI(
6             model=model or settings.LLM_MODEL_NAME,
7             google_api_key=settings.GEMINI_API_KEY,
8             temperature=temperature or settings.LLM_TEMPERATURE,
9         )
10    elif provider == 'groq':
11        return ChatGroq(
12            model=model or 'llama3-70b-8192',
13            groq_api_key=settings.GROQ_API_KEY,
14        )
15    # ... fallback chain
```

Listing 7.3: LLM Provider Factory (career/services/llm_provider.py)

7.1.4 FAISS Vector Database

The vector database uses HNSW (Hierarchical Navigable Small World) indexing, which enables efficient approximate nearest-neighbour search:

```
1 class FAISSVectorDB:
2     def __init__(self, dimension=384, index_path=None):
3         self.dimension = dimension
4         self._lock = threading.Lock() # Thread safety
5         self.metadata = []
6         self.id_to_idx = {}
7         if not self._load_index():
8             self._create_empty_index()
9
10    def _create_empty_index(self):
11        self.index = faiss.IndexHNSWFlat(self.dimension, 32)
12        self.index.hnsw.efConstruction = 200
13        self.index.hnsw.efSearch = 50
14
15    def search_by_text(self, query, k=5, score_threshold=0.0):
16        svc = EmbeddingService.get_instance()
17        embedding = svc.embed_text(query)
18        return self.search(embedding, k=k, score_threshold=
            score_threshold)
```

Listing 7.4: FAISS Vector Database (core/vector_db.py)

7.1.5 WebSocket Consumers

Real-time messaging is managed by two Django Channels consumers:

- ChatConsumer: Per-conversation WebSocket for sending/receiving messages with automatic read-receipt handling.
- NotificationConsumer: Global WebSocket for cross-cutting events (new messages, online status changes, session updates).

7.1.6 Agora Video Integration

Server-side token generation keeps video sessions secure:

```
1 from agora_token_builder import RtcTokenBuilder
2
3 def generate_video_token(channel_name, uid, role):
4     token = RtcTokenBuilder.buildTokenWithUid(
5         settings.AGORA_APP_ID,
6         settings.AGORA_APP_CERTIFICATE,
7         channel_name,
```

```
8         uid,  
9         role,  
10        privilege_expired_ts=int(time.time()) + 3600  
11    )  
12    return token
```

Listing 7.5: Agora Token Generation (mentorship/services/video_service.py)

7.2 Frontend Implementation

7.2.1 Application Architecture

The React application is structured in layers:

```
1 src/  
2   api/           // Centralized Axios API functions  
3   components/    // Reusable UI components  
4   contexts/      // Global state (Auth, Career, Theme, Chat)  
5   hooks/         // Custom hooks (useAuth, useNotifications,  
6                   useWebSocket)  
7   pages/         // Route-level page components  
8   routes/        // Router config + route guards  
9   utils/         // Helper functions
```

Listing 7.6: React Application Structure

7.2.2 Axios Interceptor for Token Refresh

The API client handles transparent JWT refresh along with request queuing:

```
1 api.interceptors.response.use(  
2   (response) => response,  
3   async (error) => {  
4     const original = error.config;  
5     if (error.response?.status === 401 && !original._retry) {  
6       if (isRefreshing) {  
7         // Queue this request until refresh completes  
8         return new Promise((resolve, reject) => {  
9           pendingQueue.push({ resolve, reject });  
10          }).then(token => {  
11            original.headers.Authorization = `Bearer ${token}`;  
12            return api(original);  
13          });  
14     }  
15   });
```

```
14     }
15     isRefreshing = true;
16     original._retry = true;
17     const { data } = await refreshClient.post(REFRESH_URL);
18     localStorage.setItem('access_token', data.access_token);
19     processPendingQueue(null, data.access_token);
20     // Retry original request
21     original.headers.Authorization = `Bearer ${data.access_token}`;
22     return api(original);
23   }
24   return Promise.reject(error);
25 }
26 );
```

Listing 7.7: Axios Token Refresh Interceptor (api/client.js)

7.2.3 Theme System

The theme system works through CSS custom properties managed by React Context:

```
1  const ThemeContext = createContext();
2
3  export function ThemeProvider({ children }) {
4    const [theme, setTheme] = useState(
5      () => localStorage.getItem('theme') || 'dark'
6    );
7
8    useEffect(() => {
9      document.documentElement.setAttribute('data-theme', theme);
10     localStorage.setItem('theme', theme);
11   }, [theme]);
12
13   return (
14     <ThemeContext.Provider value={{ theme, toggleTheme }}>
15       {children}
16     </ThemeContext.Provider>
17   );
18 }
```

Listing 7.8: Theme Context Implementation

7.2.4 WebSocket Hook for Real-Time Features

```
1 export function useNotifications() {
2   const [ws, setWs] = useState(null);
3
4   useEffect(() => {
5     const token = localStorage.getItem('access_token');
6     const socket = new WebSocket(
7       `ws://localhost:8000/ws/notifications/?token=${token}`
8     );
9     socket.onmessage = (event) => {
10      const data = JSON.parse(event.data);
11      switch (data.type) {
12        case 'new_message': updateUnreadCount(data); break;
13        case 'online_status': updateOnlineUsers(data); break;
14        case 'session_update': handleSessionUpdate(data); break;
15      }
16    };
17    setWs(socket);
18    return () => socket.close();
19  }, []);
20 }
```

Listing 7.9: Notification WebSocket Hook (hooks/useNotifications.js)

Chapter 8

Testing

8.1 Testing Approach and Strategy

8.1.1 Testing Framework

The project uses Django's built-in `TestCase` framework together with Django REST Framework's `APIClient` for comprehensive API testing. The main testing tools include:

- **Django TestCase:** Provides isolated test databases, transaction rollback, and assertion helpers.
- **DRF APIClient:** Simulates HTTP requests to API endpoints with authentication support.
- **Factory Methods:** Custom helper methods in test classes for creating test users, profiles, and related data.
- **Coverage:** All 7 Django apps covered with unit and integration tests.

8.1.2 Testing Strategy

The testing strategy takes a **bottom-up approach**:

1. **Unit Tests:** Individual model methods, service functions, validators, and serializers.
2. **Integration Tests:** End-to-end API endpoint testing including authentication, authorization, database operations, and response validation.
3. **Security Tests:** Password hashing verification, OTP hash verification, authorization boundary tests, account lockout flows.
4. **Edge Case Tests:** Missing fields, invalid data, expired tokens, duplicate entries, rate limit scenarios.

8.2 Item Pass/Fail Criteria

- **Pass:** The test assertion succeeds, the expected HTTP status code is returned, and the response body matches the expected schema.
- **Fail:** Any assertion error, unexpected status code, incorrect response structure, or unhandled exception during test execution.
- **Error:** Test infrastructure failure (database connection, import error) preventing test execution.

8.3 Test Results Summary

Table 8.1: Overall Test Execution Results

Metric	Value
Total Tests	138
Passed	138
Failed	0
Errors	0
Skipped	0
Execution Time	~80 seconds
Result	ALL TESTS PASS

Table 8.2: Test Distribution by Application

App	Tests	Status	SRS Test Cases Covered
accounts	47	All Pass	TC-1, TC-2, TC-11, TC-17, TC-18, TC-20
career	26	All Pass	TC-3, TC-4, TC-5, TC-15, TC-20
mentorship	23	All Pass	TC-7, TC-8, TC-10, TC-12
profiles	20	All Pass	TC-2, TC-14
chatbot	12	All Pass	TC-6, TC-20
messaging	10	All Pass	TC-9

8.4 Detailed Test Cases

8.4.1 Authentication Tests (accounts – 47 tests)

#	Test Name	Description	Endpoint	Result
1	test_signup_success	Valid signup creates user and OTP	POST /auth/signup	
2	test_signup_duplicate_email	Duplicate email returns 400	POST /auth/signup	
3	test_signup_weak_password	Weak password returns 400	POST /auth/signup	
4	test_signup_missing_name	Missing full_name returns 400	POST /auth/signup	
5	test_verify_otp_success	Correct OTP verifies user	POST /auth/verify-otp	
6	test_verify_otp_wrong	Wrong OTP returns 400	POST /auth/verify-otp	
7	test_verify_otp_expired	Expired OTP returns 400	POST /auth/verify-otp	
8	test_resend_otp	Resend creates new OTP	POST /auth/resend-otp	
9	test_signin_success	Valid credentials return tokens	POST /auth/signin	
10	test_signin_wrong_pwd	Wrong password returns 401	POST /auth/signin	
11	test_signin_unverified	Unverified user returns 403	POST /auth/signin	
12	test_user_me	Authenticated user data	GET /auth/me	
13	test_me_unauthenticated	No token returns 401	GET /auth/me	
14	test_dashboard	Dashboard stats returned	GET /auth/dashboard	
15	test_lockout_after_max5	5 failures lock account	POST /auth/signin	
16	test_lockout_correct_pwd	Locked account rejects login	POST /auth/signin	
17	test_lockout_expires	Lock expires, login succeeds	POST /auth/signin	
18	test_failed_login_record	Failed login creates record	POST /auth/signin	
19	test_admin_dashboard	Admin access succeeds	GET /admin/dashboard/	
20	test_non_admin_denied	Non-admin returns 403	GET /admin/dashboard/	
25	test_tokens_returned	Signin returns both tokens	POST /auth/signin	
26	test_refresh_rotates	Refresh generates new token	POST /auth/refresh	
28	test_logout_clears	Logout clears cookie	POST /auth/logout	
31	test_pwd_hashed	Password stored as hash	DB Check	
32	test_otp_hashed	OTP stored as SHA-256	DB Check	

(Remaining 22 authentication tests cover password validation, change password flow, forgot/reset password, error handling edge cases – all passing.)

8.4.2 Career Planning Tests (career – 26 tests)

#	Test Name	Description	Endpoint	Result
1	test_start_profile	Creates new career profile	POST /career/start/	
2	test_resume_profile	Resumes in-progress profile	GET /career/resume/	
3	test_submit_step_1	Submits phase 1 data	PATCH /career/{id}/step/	
4	test_submit_all_steps	Completes all 6 steps	PATCH (x6)	
5	test_finish_profile	Triggers plan generation	POST /career/{id}/finish/	

#	Test Name	Description	Endpoint	Result
6	test_get_plan	Retrieves generated plan	GET /career/{id}/plan/	
7	test_plan_status	Gets plan generation status	GET /career/{id}/plan/status/	
8	test_completeness	Calculates profile completeness	GET /career/{id}/completeness/	
9	test_step_schema	Returns step schema	GET /career/step/{n}/schema/	
10	test_invalid_step	Rejects step > 6	PATCH /career/{id}/step/	

(Remaining 16 tests cover plan refinement, error handling, concurrent access, permissions – all passing.)

8.4.3 Mentorship Tests (mentorship – 23 tests)

Key test scenarios:

- Mentor profile CRUD operations (create, read, update)
- Mentor discovery with search and filtering
- Connection request lifecycle (send, accept, reject, cancel)
- Session scheduling and status transitions
- Post-session feedback submission and validation
- Video token generation
- Permissions enforcement (only participants can access sessions)

8.4.4 Profiles Tests (profiles – 20 tests)

Key test scenarios:

- Professional profile creation and retrieval
- Skill CRUD with proficiency validation
- Work experience with date validation
- Education entry management
- Achievement and certification tracking
- Top strength and improvement area ordering
- Profile completeness calculation
- Ownership enforcement across all sub-resources

8.4.5 Chatbot Tests (chatbot – 12 tests)

Key test scenarios:

- Chat session creation with various types
- Message sending and AI response generation
- Chat history retrieval with pagination
- Session detail retrieval and deletion
- Ownership validation (users can only access own sessions)

8.4.6 Messaging Tests (messaging – 10 tests)

Key test scenarios:

- Conversation creation between participants
- Message sending via REST API
- Mark-as-read functionality
- Total unread count endpoint
- Conversation list ordering by latest message
- Participant-only access enforcement

8.5 Bugs Found and Fixed During Testing

Table 8.5: Bugs Discovered and Resolved During Testing

#	Severity	Issue	Resolution
1	High	CareerPlan.plan_type default 'comprehensive' not in PlanType enum	Changed default to PlanType.INITIAL
2	Medium	Frontend called PUT/DELETE on /profiles/strengths/<id>/ but no backend route existed	Added TopStrengthDetailAPIView and ImprovementAreaDetailAPIView with URL patterns
3	Medium	Completeness serializer expected different field names than service provided	Aligned serializer field names with service output
4	Medium	GetPlanAPIView used .get() risking MultipleObjectsReturned	Changed to .filter().first() with null check

8.6 Risks and Contingencies

Table 8.6: Testing Risks and Mitigation Strategies

Risk	Impact	Mitigation
LLM API rate limits	Career plan generation tests may fail intermittently	Mock LLM provider (LLM_PROVIDER=mock) for deterministic tests
Redis unavailability	Token blacklisting and caching may fail	Graceful fallback to in-memory cache; tests run without Redis
WebSocket testing	Django test client doesn't support WebSocket	WebSocket consumers tested via Channels test utilities
Database migrations	Schema changes may break existing test data	Fresh test database per run; migration verification in CI
External API changes	Google OAuth, Agora API changes	Mock external services in tests; version-pin SDKs

Chapter 9

Deployment and DevOps

9.1 Docker Compose Architecture

The application runs in containers managed by Docker Compose, with a total of 6 services:

```
1 services:
2   postgres:      PostgreSQL 15 Alpine (Port 5432)
3   redis:         Redis 7 Alpine (Port 6379)
4   app:           Django + Daphne ASGI (Port 8000)
5   celery:        Celery Worker
6   celery-beat:   Celery Beat Scheduler
7   flower:        Celery Monitoring (Port 5555)
```

Listing 9.1: Docker Compose Services

9.2 Deployment Steps

1. Clone Repository:

```
1 git clone https://github.com/Rafid13iit/Career-Talk.git
2 cd career-talk
```

2. Backend Setup:

```
1 cd backend
2 cp .env.example .env      # Configure environment variables
3 docker-compose up -d      # Start all services
4 docker-compose exec app python manage.py migrate
5 docker-compose exec app python manage.py createsuperuser
```

3. Frontend Setup:

```
1 cd frontend
2 cp .env.example .env      # Set VITE_GOOGLE_CLIENT_ID
3 npm install
4 npm run dev                # Development server on port 5173
5 npm run build              # Production build
```

9.3 Environment Configuration

Below are the key environment variables (set through the `.env` file):

Table 9.1: Key Environment Variables

Variable	Default	Description
ENV	development	Environment mode
SECRET_KEY	(random)	Django secret key
DB_NAME	career_talk	PostgreSQL database name
LLM_PROVIDER	gemini	AI provider (gemini/groq/mock)
GEMINI_API_KEY	(empty)	Google Gemini API key
AGORA_APP_ID	(empty)	Agora RTC application ID
GOOGLE_CLIENT_ID	(empty)	Google OAuth client ID
USE_REDIS	false	Enable Redis (true in production)

Chapter 10

User Manual

10.1 System Requirements

Table 10.1: Minimum System Requirements

Component	Requirement
Browser	Chrome 90+, Firefox 90+, Safari 15+, Edge 90+
Internet	Stable broadband connection (for video calls: 2+ Mbps)
Screen Resolution	Minimum 320px width (mobile responsive)
JavaScript	Must be enabled
Cookies	Must be enabled (for JWT refresh tokens)

10.2 Getting Started

10.2.1 Registration

1. Open `http://localhost:5173` (or the deployed URL) in your browser.
2. Click **“Sign Up”** on the landing page or in the navigation bar.
3. Fill in your full name, email address, and a strong password (at least 8 characters, including uppercase, lowercase, and numbers).
4. You can also click **“Sign in with Google”** for a quicker registration.
5. If you registered with email, check your inbox for a 6-digit OTP.
6. Enter the OTP within 10 minutes to verify your account.

10.2.2 Sign In

1. Click **“Sign In”** and type in your credentials.
2. Once authenticated, you will be taken to the Dashboard.
3. Your session stays active for 30 days thanks to the refresh token stored in an HttpOnly cookie.

10.3 Dashboard

The Dashboard gives you an overview of your career journey:

- **Career Score:** An AI-calculated career readiness score ranging from 0 to 100.
- **Profile Completeness:** Shows what percentage of your professional profile has been filled.
- **Quick Actions:** Shortcuts to the career wizard, AI chatbot, and mentor discovery.
- **Recent Activity:** Your latest sessions, messages, and plan updates.

10.4 Career Wizard

1. Go to “**Career Wizard**” from the Dashboard or the sidebar.
2. Work through the 6 phases of career profiling:

Phase 1: **Basic Information:** Name, age, location, career status.

Phase 2: **Education:** Degrees, institutions, fields of study.

Phase 3: **Skills & Capabilities:** Technical and soft skills with proficiency.

Phase 4: **Experience:** Work history, projects, internships.

Phase 5: **Goals & Aspirations:** Short-term and long-term career goals.

Phase 6: **Preferences:** Work style, location, salary expectations.

3. Progress is auto-saved, so you can leave and resume anytime.
4. After finishing all six phases, click “**Generate Career Plan**”.
5. The AI will process your profile (this usually takes 15 to 60 seconds) and create a personalized career plan.

10.5 AI Career Plan

The generated career plan covers the following areas:

- **Career Tracks:** Two to three recommended career paths, each with descriptions and suitability scores.
- **Skill Gaps:** Missing skills identified by priority level, with suggested resources for each.

- **Professional Roadmap:** A time-phased action plan spanning 3 to 12 months, with milestones.
- **Resources:** Courses, certifications, books, and tools for skill development.
- **Opportunities:** Job roles, companies, and industries that align with your profile.

You can refine the plan by sharing feedback on specific sections.

10.6 AI Chatbot

1. Click the chat icon in the bottom-right corner or go to the AI Mentor page.
2. Start a new conversation or pick up where you left off in an existing session.
3. Ask career-related questions. The AI uses your profile data and career knowledge base for context-aware responses.
4. Some example questions: “What skills should I learn for data science?”, “How to transition from backend to full-stack?”

10.7 Finding and Connecting with Mentors

1. Go to the “**Experts**” page.
2. Browse through mentor cards that display expertise, rating, experience, and availability.
3. Use the **search bar** and **filters** (expertise area, industry, price range, rating) to narrow down your options.
4. Click “**Connect**” on a mentor’s card and include a personalized message.
5. The mentor will review your request and respond accordingly.

10.8 Scheduling Sessions

1. After connecting with a mentor, go to “**Session Detail**”.
2. Click “**Schedule Session**” and pick:
 - Session type: Video, Audio, or Chat.
 - A date and time that works based on the mentor’s availability.
 - Duration: 30, 45, or 60 minutes.

3. When it is time for the session, click **“Join Session”** to enter the video room.
4. Afterwards, you can submit feedback with a 1 to 5 star rating and optional comments.

10.9 Messaging

1. Open **“Messages”** from the sidebar.
2. Pick a conversation from the left panel.
3. Type and send messages in real time.
4. A green dot next to a name means that person is currently online.
5. The navigation bar shows a badge with the number of unread messages.

10.10 Becoming a Mentor

1. Go to **“Mentor Dashboard”** from the sidebar.
2. Click **“Create Mentor Profile”** and fill in:
 - Headline, bio, and expertise areas.
 - Industries, years of experience, and hourly rate.
 - Languages spoken and timezone.
3. Set your weekly **availability** (day, start time, end time).
4. Manage incoming **connection requests** (accept/reject).
5. View your **mentor statistics**: total sessions, hours, ratings.

10.11 Theme Switching

Click the **sun/moon icon** in the navigation bar to switch between Light and Dark modes. Your preference is saved automatically and will persist across sessions.

Chapter 11

Conclusion and Future Work

11.1 Summary of Achievements

Career Talk demonstrates that it is feasible to build an integrated AI-powered career guidance platform. The main accomplishments include:

- **Full-Stack Implementation:** A production-ready application with 67+ API endpoints, 7 Django apps, and 18+ frontend pages.
- **AI Integration:** A multi-agent LLM pipeline for career plan generation, paired with FAISS-based RAG for knowledge retrieval.
- **Real-Time Features:** WebSocket-powered messaging and notifications, along with Agora video conferencing.
- **Comprehensive Security:** JWT with rotation, SHA-256 OTP hashing, account lockout, and role-based access control.
- **User Experience:** A responsive design with dark/light themes, animations, and intuitive navigation.
- **Quality Assurance:** 138 automated tests with a 100% pass rate across all application modules.
- **DevOps Ready:** Docker Compose deployment bundling PostgreSQL, Redis, Celery, and health monitoring.

11.2 Challenges Faced

Several challenges came up during the development process:

1. **LLM Response Consistency:** Getting structured JSON output from LLMs required carefully written prompts and robust parsing logic with fallback mechanisms.
2. **WebSocket Authentication:** Adding JWT-based WebSocket authentication in Django Channels meant writing custom middleware to extract tokens from query parameters.

3. **Token Refresh Race Conditions:** When multiple concurrent requests got 401 responses, they could all try to refresh the token at the same time. This was solved with a queuing mechanism in the Axios interceptor.
4. **Model Size Constraints:** Sentence-transformer models and PyTorch dependencies increased the Docker image size considerably. Multi-stage Docker builds helped mitigate this.
5. **Theme Consistency:** Keeping the visual appearance consistent across dark and light themes for 18+ pages required careful management of CSS custom properties.

11.3 Limitations

- Right now, the platform supports only English. Multi-language support is planned for a future release.
- Payment integration for premium mentorship sessions has not been implemented yet.
- The FAISS knowledge base needs periodic manual updates with new career data.
- Mobile app (React Native) is not available yet. The platform is currently web-only with responsive design.
- AI responses rely on third-party LLM APIs (Gemini/Groq), which can have availability and rate-limit constraints.

11.4 Future Work

There are several directions this project could take going forward:

1. **Mobile Application:** A React Native app for iOS and Android with push notifications.
2. **Payment Gateway:** Stripe or bKash integration for paid mentorship sessions.
3. **Multi-Language Support:** Bengali, Hindi, and Urdu interfaces to reach a broader audience.
4. **Resume Builder:** AI-powered resume generation based on professional profile data.
5. **Job Board Integration:** Direct job application links drawn from career plan recommendations.
6. **AI Model Fine-Tuning:** Custom fine-tuned models trained on South Asian career data.
7. **Analytics Dashboard:** More detailed analytics for tracking career progress over time.

8. **Peer Networking:** User-to-user connections and group discussion features.
9. **Gamification:** Badges, streaks, and achievements for reaching career milestones.
10. **Offline Mode:** A Progressive Web App (PWA) with offline access to career plans.

11.5 Concluding Remarks

Career Talk shows that AI-powered career guidance tools can be built to be accessible, personalized, and thorough. By combining the analytical strength of large language models with the irreplaceable value of human mentorship, the platform creates a balanced approach to career development. Its modular architecture, comprehensive test suite, and modern technology stack mean the platform is well-positioned to scale and evolve as it serves a growing user base across South Asia and beyond.

References

- [1] Django Software Foundation, “Django Documentation,” 2024. [Online]. Available: <https://docs.djangoproject.com/en/5.0/>
- [2] Django REST Framework, “DRF Documentation,” 2024. [Online]. Available: <https://www.django-rest-framework.org/>
- [3] Meta AI, “Facebook AI Similarity Search (FAISS),” 2024. [Online]. Available: <https://github.com/facebookresearch/faiss>
- [4] LangChain, “LangChain Documentation,” 2024. [Online]. Available: <https://python.langchain.com/docs/>
- [5] Google, “Gemini API Documentation,” 2024. [Online]. Available: <https://ai.google.dev/docs>
- [6] React Team, “React Documentation,” 2024. [Online]. Available: <https://react.dev/>
- [7] Tailwind Labs, “Tailwind CSS Documentation,” 2024. [Online]. Available: <https://tailwindcss.com/docs>
- [8] Agora.io, “Agora RTC SDK Documentation,” 2024. [Online]. Available: <https://docs.agora.io/en/>
- [9] Django Channels, “Channels Documentation,” 2024. [Online]. Available: <https://channels.readthedocs.io/en/stable/>
- [10] Sentence Transformers, “Sentence-BERT Documentation,” 2024. [Online]. Available: <https://www.sbert.net/>
- [11] Lewis, P. et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *NeurIPS*, 2020.
- [12] Vaswani, A. et al., “Attention is All You Need,” *NeurIPS*, 2017.
- [13] Docker Inc., “Docker Compose Documentation,” 2024. [Online]. Available: <https://docs.docker.com/compose/>
- [14] Celery Project, “Celery Documentation,” 2024. [Online]. Available: <https://docs.celeryq.dev/en/stable/>
- [15] UNESCO, “Global Education Monitoring Report: Technology in Education,” 2023. [Online]. Available: <https://www.unesco.org/gem-report/en>

- [16] Simple JWT, “Django REST Framework Simple JWT,” 2024. [Online]. Available: <https://django-rest-framework-simplejwt.readthedocs.io/>
- [17] Redis Ltd., “Redis Documentation,” 2024. [Online]. Available: <https://redis.io/documentation>
- [18] PostgreSQL Global Development Group, “PostgreSQL Documentation,” 2024. [Online]. Available: <https://www.postgresql.org/docs/>

Appendix A

API Endpoint Reference

A.1 Authentication Endpoints

Method	Path	Description	Auth
POST	/api/v1/auth/signup	User registration	Public
POST	/api/v1/auth/verify-otp	OTP verification	Public
POST	/api/v1/auth/resend-otp	Resend OTP	Public
POST	/api/v1/auth/signin	User login	Public
POST	/api/v1/auth/google	Google OAuth	Public
POST	/api/v1/auth/refresh	Refresh JWT	Cookie
POST	/api/v1/auth/logout	User logout	Auth
GET	/api/v1/auth/me	Current user info	Auth
POST	/api/v1/auth/forgot-password	Request reset OTP	Public
POST	/api/v1/auth/reset-password	Reset password	Public
POST	/api/v1/auth/change-password	Change password	Auth
GET	/api/v1/auth/dashboard	User dashboard stats	Auth

A.2 Career Planning Endpoints

Method	Path	Description	Auth
POST	/api/v1/career/start/	Start career profile	Auth
GET	/api/v1/career/resume/	Resume in-progress profile	Auth
GET	/api/v1/career/{id}/	Profile detail	Owner
DELETE	/api/v1/career/{id}/	Delete profile	Owner
POST	/api/v1/career/{id}/finish/	Finish & trigger generation	Owner
GET	/api/v1/career/{id}/completeness/	Completeness report	Owner
GET	/api/v1/career/step/{n}/schema/	Get step N schema	Auth
PATCH	/api/v1/career/{id}/step/	Submit step data	Owner
GET	/api/v1/career/{id}/plan/	Get generated plan	Owner
GET	/api/v1/career/{id}/plan/status/	Plan generation status	Owner
POST	/api/v1/career/{id}/plan/refine/	Refine existing plan	Owner

A.3 Mentorship Endpoints

Method	Path	Description	Auth
POST/GET/PUT	/api/v1/mentorship/mentor/profile/	Mentor profile CRUD	Auth
GET	/api/v1/mentorship/mentor/dashboard/	Mentor dashboard	Mentor
GET/POST	/api/v1/mentorship/mentor/availability/	Availability slots	Mentor
PUT/DELETE	/api/v1/mentorship/mentor/availability/{id}/	Slot detail	Mentor
GET	/api/v1/mentorship/mentors/	Discover mentors	Auth
GET	/api/v1/mentorship/mentors/{id}/	Mentor public profile	Auth
POST/GET	/api/v1/mentorship/connections/	Connection requests	Auth
POST	/api/v1/mentorship/connections/{id}/respond/	Accept/reject	Mentor
GET/POST	/api/v1/mentorship/sessions/	Session list/create	Auth
GET/PATCH	/api/v1/mentorship/sessions/{id}/	Session detail/update	Participant
POST	/api/v1/mentorship/sessions/{id}/start/	Start session	Mentor
POST	/api/v1/mentorship/sessions/{id}/end/	End session	Mentor
POST	/api/v1/mentorship/sessions/{id}/feedback/	Submit feedback	Participant
POST	/api/v1/mentorship/sessions/{id}/video-token/	Get RTC token	Participant

A.4 Chatbot Endpoints

Method	Path	Description	Auth
GET/POST	/api/v1/chatbot/sessions/	List/create sessions	Auth
GET/DELETE	/api/v1/chatbot/sessions/{id}/	Session detail/delete	Owner
POST	/api/v1/chatbot/messages/send/	Send message & get AI reply	Auth
GET	/api/v1/chatbot/sessions/{id}/messages/	Message history	Owner

A.5 Messaging & WebSocket Endpoints

Method	Path	Description	Auth
GET	/api/v1/messaging/conversations/	List conversations	Auth
POST	/api/v1/messaging/conversations/	Create conversation	Auth
GET	/api/v1/messaging/conversations/{id}/messages/	Message history	Auth
POST	/api/v1/messaging/conversations/{id}/read/	Mark as read	Auth
GET	/api/v1/messaging/unread/total/	Total unread count	Auth
WebSocket Endpoints			
WS	ws/chat/{conversation_id}/?token={jwt}	Per-conversation chat	JWT

Method	Path	Description	Auth
WS	ws/notifications/?token={jwt}	Global notifications	JWT

Appendix B

Database Schema Details

B.1 Complete Table Definitions

Column	Type	Constraints	Description
Table: users			
id	UUID	PK, auto-generated	Unique identifier
email	VARCHAR(320)	UNIQUE, NOT NULL	Login email
full_name	VARCHAR(100)	NULLABLE	User's display name
password	VARCHAR	Hashed	Argon2 hashed password
role	VARCHAR(20)	DEFAULT 'user'	'user' or 'admin'
google_id	VARCHAR(100)	UNIQUE, NULLABLE	Google OAuth ID
avatar_url	URL	NULLABLE	Profile picture URL
is_active	BOOLEAN	DEFAULT FALSE	Account activated
is_verified	BOOLEAN	DEFAULT FALSE	Email verified via OTP
failed_login_attempts	INTEGER	DEFAULT 0	Lockout counter
locked_until	DATETIME	NULLABLE	Account lock expiry
created_at	DATETIME	Auto	Registration timestamp
updated_at	DATETIME	Auto	Last update timestamp
Table: career_profiles			
id	UUID	PK	Unique identifier
user_id	UUID	FK -> users	Profile owner
status	VARCHAR(20)	INDEXED	Profile status enum
current_step	INTEGER	DEFAULT 1	Current wizard step (1–6)
phase_1_basic	JSON	DEFAULT {}	Basic info data
phase_2_education	JSON	DEFAULT {}	Education data
phase_3_skills	JSON	DEFAULT {}	Skills data
phase_4_experience	JSON	DEFAULT {}	Experience data
phase_5_goals	JSON	DEFAULT {}	Goals data
phase_6_preferences	JSON	DEFAULT {}	Preferences data
completed_at	DATETIME	NULLABLE	Completion timestamp
Table: career_plans			
id	UUID	PK	Unique identifier
profile_id	UUID	FK -> career_profiles	Source profile
user_id	UUID	FK -> users	Plan owner

Column	Type	Constraints	Description
Table: users			
status	VARCHAR(20)	INDEXED	Generation status
plan_type	VARCHAR(50)	DEFAULT 'initial'	initial or refined
summary	TEXT	NULLABLE	Executive summary
recommended_tracks	JSON	DEFAULT []	Career track objects
skill_gaps	JSON	DEFAULT []	Skill gap objects
roadmap	JSON	DEFAULT []	Roadmap phase objects
resources	JSON	DEFAULT []	Resource objects
model_used	VARCHAR(100)	NULLABLE	LLM model name
processing_time_seconds	FLOAT	NULLABLE	Generation time

Appendix C

Project Directory Structure

```
1 Career Talk/
2 |-- CLAUDE.md                                # AI context document
3 |-- backend/
4 |   |-- manage.py                            # Django management script
5 |   |-- requirements.txt                     # 228 Python dependencies
6 |   |-- docker-compose.yml                  # 6-service Docker deployment
7 |   |-- Dockerfile                          # Django container image
8 |   |-- .env / .env.example                 # Environment config
9 |   |-- backend/
10 |      |-- settings.py                      # 390-line Django settings
11 |      |-- urls.py                          # Root URL config (8 app includes)
12 |      |-- celery.py                       # Celery app config
13 |      |-- asgi.py / wsgi.py               # Server gateways
14 |      |-- accounts/                       # Auth + user management (47 tests)
15 |      |-- models.py                       # User, RefreshToken, OTP, FailedLogin
16 |      |-- views/                         # auth, oauth, password, dashboard, admin
17 |      |-- services/                      # auth_service, email_service, token_service
18 |      |-- profiles/                      # Professional profiles (20 tests)
19 |      |-- models.py                       # Profile, Skill, WorkExp, Education, etc.
20 |      |-- career/                        # Career planning (26 tests)
21 |      |-- models.py                       # CareerProfile, CareerPlan, PlanEvaluation
22 |      |-- services/                      # career_agent, llm_provider, completeness
23 |      |-- chatbot/                       # AI chatbot (12 tests)
24 |      |-- models.py                       # ChatSession, ChatMessage
25 |      |-- views/                         # sessions, messages
26 |      |-- mentorship/                   # Mentorship system (23 tests)
27 |      |-- models.py                       # MentorProfile, Session, Feedback
28 |      |-- services/                      # mentor_service, video_service
29 |      |-- messaging/                     # Real-time messaging (10 tests)
30 |      |-- consumers.py                   # ChatConsumer, NotificationConsumer
31 |      |-- routing.py                     # WebSocket URL patterns
32 |      |-- core/                          # Shared utilities
33 |      |-- vector_db.py                   # FAISSVectorDB, CareerKnowledgeBase
34 |      |-- embeddings.py                  # EmbeddingService (sentence-transformers)
35 |      |-- data/                          # FAISS index files
36 |-- frontend/
37 |   |-- package.json                       # 25+ dependencies
38 |   |-- vite.config.js                     # Vite build config
39 |   |-- src/
40 |      |-- App.jsx                         # Root component
41 |      |-- api/                           # 8 API modules (auth, career, mentor, etc.)
42 |      |-- contexts/                      # Auth, Career, Theme, Chat
43 |      |-- pages/                         # 18 page components
44 |      |-- components/                    # Reusable UI + animations
45 |      |-- hooks/                         # Custom hooks (auth, WS, notifications)
46 |      |-- routes/                        # AppRouter + RouteGuards
```

Listing C.1: Complete Project Directory Structure